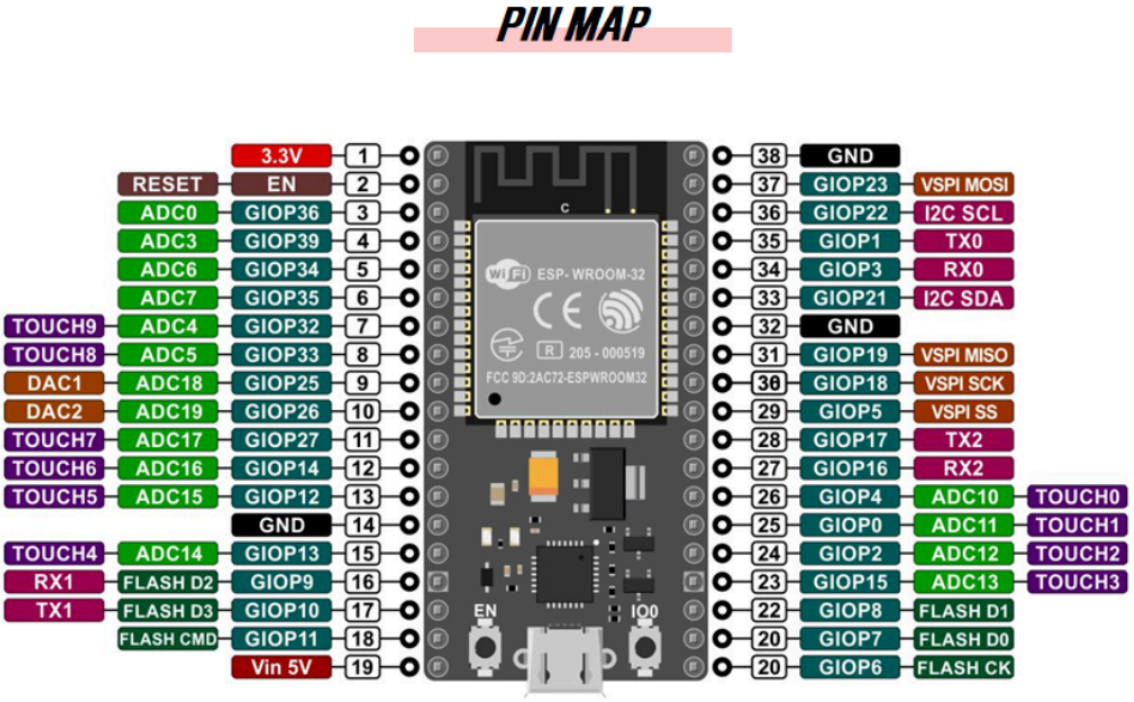
ESP32, Arduino

진행환경

하드웨어 : 2018맥프로

IDE : ArduinoIDE

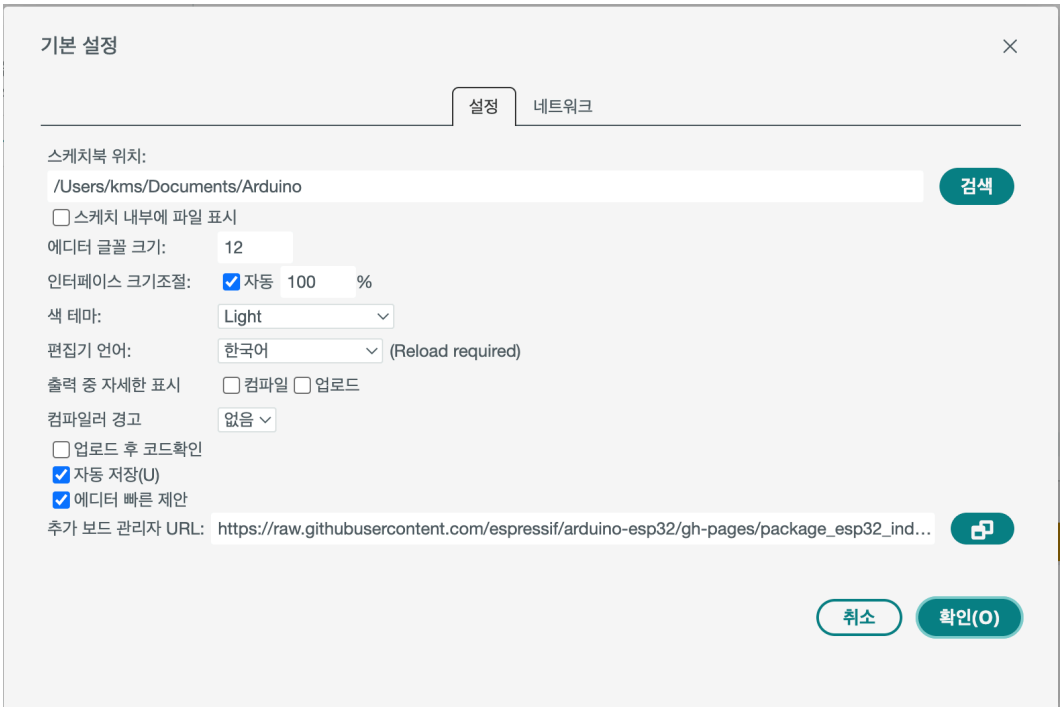
Module : ESP32 CP2102



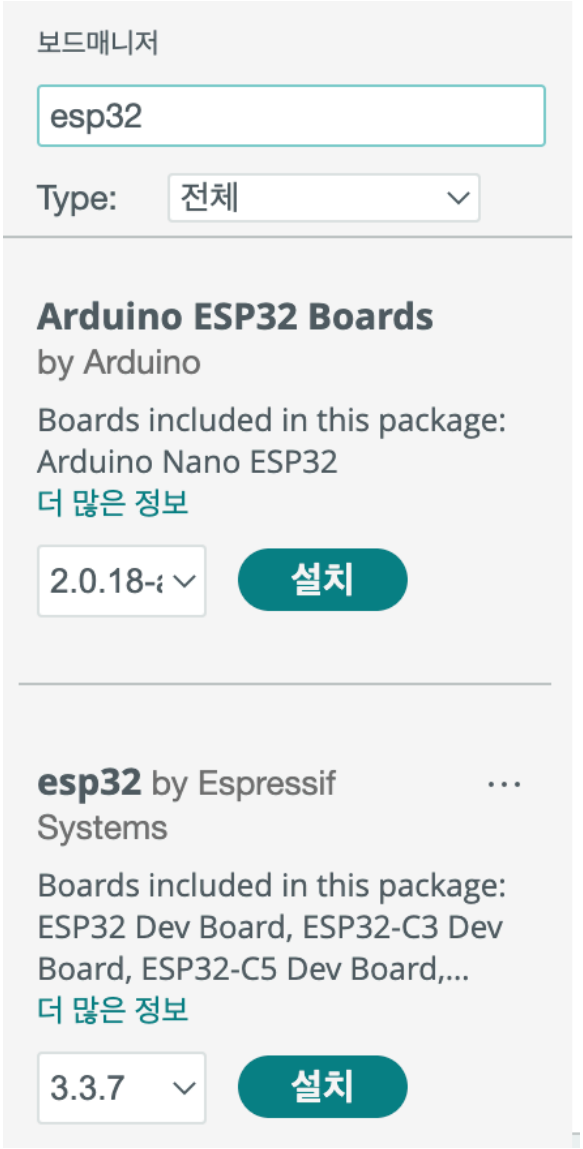
위 그림상의 USB커넥터는 예시이고 Type-C USB로 출고 됩니다.

상단메뉴 → 기본설정 → 추가 보드 관리자 URL

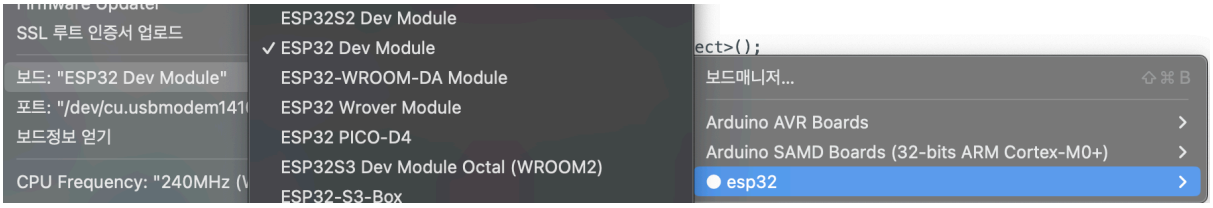
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json



도구 → 보드 → 보드 매니저 → 검색 (ESP32) → “esp32 by Espressif” 설치



도구 → 보드 → esp32 → ESP32 Dev Module 선택



기본 예제코드

https://github.com/espressif/arduino-esp32

▼ server측

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
#include <BLE2902.h>

BLECharacteristic *pCharacteristic;
bool deviceConnected = false;

// ✅ 조도센서 핀 추가 (DO 연결한 핀)
int sensorPin = 25;

// UUID (클라이언트와 반드시 동일)
#define SERVICE_UUID        "12345678-1234-1234-1234-123
4567890ab"
#define CHARACTERISTIC_UUID "abcd1234-5678-1234-5678-123
4567890ab"

// 연결 상태 콜백
class MyServerCallbacks: public BLEServerCallbacks {
  void onConnect(BLEServer* pServer) {
    deviceConnected = true;
    Serial.println(">>> Client Connected");
  }

  void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    Serial.println(">>> Client Disconnected");
    BLEDevice::startAdvertising();
  }
};

void setup() {
  Serial.begin(115200);
  delay(1000);
```

```
  Serial.println("===== SERVER START =====");

  // ✅ 센서 핀 입력 설정
  pinMode(sensorPin, INPUT);

  // BLE 초기화
  BLEDevice::init("ESP32_BLE_SERVER");

  BLEServer *pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService = pServer->createService(SERVICE_
UUID);

  pCharacteristic = pService->createCharacteristic(
                      CHARACTERISTIC_UUID,
                      BLECharacteristic::PROPERTY_NOTIFY
                    );

  pCharacteristic->addDescriptor(new BLE2902());
  pService->start();

  BLEAdvertising *pAdvertising = BLEDevice::getAdvertisi
ng();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);

  BLEDevice::startAdvertising();
  Serial.println(">>> BLE Advertising Started");
}

void loop() {

  // ✅ 조도센서 값 읽기
  int lightValue = digitalRead(sensorPin);

  // 시리얼 출력
  Serial.print("Light: ");
```

```
  Serial.println(lightValue);  // 0 또는 1

  if (deviceConnected) {
    // 문자열로 변환해서 전송
    String msg = String(lightValue);

    pCharacteristic->setValue(msg.c_str());
    pCharacteristic->notify();

    Serial.print(">>> Sent: ");
    Serial.println(msg);
  }

  delay(1000);  // 1초마다
}
```

▼ client측

```
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEClient.h>
#include <BLERemoteCharacteristic.h>
#include <BLEAdvertisedDevice.h>

#define SERVICE_UUID        "12345678-1234-1234-1234-123
4567890ab"
#define CHARACTERISTIC_UUID "abcd1234-5678-1234-5678-123
4567890ab"

static BLERemoteCharacteristic* pRemoteCharacteristic;
static BLEAdvertisedDevice* myDevice;

bool doConnect = false;
bool connected = false;

// 🔥 모든 BLE 장치 출력 + 이름으로 필터링
class MyAdvertisedDeviceCallbacks: public BLEAdvertisedD
eviceCallbacks {
```

```
  void onResult(BLEAdvertisedDevice advertisedDevice) {

    Serial.println("---- FOUND DEVICE ----");
    Serial.println(advertisedDevice.toString().c_str());

    // 👉 이름으로 찾기 (중요)
    if (advertisedDevice.getName() == "ESP32_BLE_SERVE
R") {
      Serial.println(">>> TARGET FOUND <<<");

      myDevice = new BLEAdvertisedDevice(advertisedDevic
e);
      doConnect = true;

      BLEDevice::getScan()->stop();
    }
  }
};

// 🔥 notify 수신 콜백
void notifyCallback(
  BLERemoteCharacteristic* pBLERemoteCharacteristic,
  uint8_t* pData,
  size_t length,
  bool isNotify) {

  // 🔥 수신 데이터 문자열로 변환
  String received = "";
  for (int i = 0; i < length; i++) {
    received += (char)pData[i];
  }

  Serial.print(">>> Raw: ");
  Serial.println(received);

  // 🔥 값 해석
  if (received == "0") {
    Serial.println(">>> 상태: 어두움");
```

```
  }
  else if (received == "1") {
    Serial.println(">>> 상태: 밝음");
  }
  else {
    Serial.println(">>> 알 수 없는 값");
  }
}

// 🔥 서버 연결
bool connectToServer() {

  Serial.println("Connecting to server...");

  BLEClient* pClient = BLEDevice::createClient();
  pClient->connect(myDevice);

  Serial.println("Connected!");

  BLERemoteService* pRemoteService = pClient->getService
(SERVICE_UUID);

  if (pRemoteService == nullptr) {
    Serial.println("❌ Service NOT found");
    return false;
  }

  Serial.println("Service found");

  pRemoteCharacteristic = pRemoteService->getCharacteris
tic(CHARACTERISTIC_UUID);

  if (pRemoteCharacteristic == nullptr) {
    Serial.println("❌ Characteristic NOT found");
    return false;
  }

  Serial.println("Characteristic found");
```

```
  // notify 등록
  pRemoteCharacteristic->registerForNotify(notifyCallbac
k);

  connected = true;
  Serial.println(">>> READY TO RECEIVE <<<");

  return true;
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("===== CLIENT START =====");

  BLEDevice::init("");

  BLEScan* pScan = BLEDevice::getScan();
  pScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDe
viceCallbacks());

  pScan->setInterval(100);
  pScan->setWindow(99);
  pScan->setActiveScan(true);

  Serial.println("Scanning...");
  pScan->start(0, nullptr); // 🔥 무한 스캔
}

void loop() {

  if (doConnect) {
    if (connectToServer()) {
      Serial.println(">>> CONNECT SUCCESS <<<");
    } else {
      Serial.println(">>> CONNECT FAIL <<<");
```

```
    }
    doConnect = false;
  }

  delay(1000);
}
```

▼ 디버깅용 모든 BLE데이터 출력

```
#include <Arduino.h>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>

BLEScan* pBLEScan;

// 모든 광고 정보를 출력하는 콜백
class MyAdvertisedDeviceCallbacks : public BLEAdvertised
DeviceCallbacks {
  void onResult(BLEAdvertisedDevice advertisedDevice) {

    Serial.println("\n===== BLE DEVICE FOUND =====");

    // 주소
    Serial.print("Address: ");
    Serial.println(advertisedDevice.getAddress().toStrin
g().c_str());

    // 이름
    Serial.print("Name: ");
    Serial.println(advertisedDevice.getName().c_str());

    // RSSI (신호 세기)
    Serial.print("RSSI: ");
    Serial.println(advertisedDevice.getRSSI());
```

```
    // Manufacturer Data (핵심)
    std::string raw = advertisedDevice.getManufacturerDa
ta();
    if (raw.length() > 0) {
      Serial.print("Manufacturer Data (ASCII): ");
      for (int i = 0; i < raw.length(); i++) {
        Serial.print((char)raw[i]);
      }
      Serial.println();

      Serial.print("Manufacturer Data (HEX): ");
      for (int i = 0; i < raw.length(); i++) {
        if ((uint8_t)raw[i] < 0x10) Serial.print("0");
        Serial.print((uint8_t)raw[i], HEX);
        Serial.print(" ");
      }
      Serial.println();
    } else {
      Serial.println("Manufacturer Data: NONE");
    }

    // Service UUID
    if (advertisedDevice.haveServiceUUID()) {
      Serial.print("Service UUID: ");
      Serial.println(advertisedDevice.getServiceUUID().t
oString().c_str());
    }

    // 전체 raw 정보
    Serial.print("RAW: ");
    Serial.println(advertisedDevice.toString().c_str());

    Serial.println("============================");
  }
};

void setup() {
```

```
  Serial.begin(115200);
  Serial.println("Starting BLE Scanner (DEBUG MOD
E)...");

  BLEDevice::init("");

  pBLEScan = BLEDevice::getScan();
  pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertise
dDeviceCallbacks());

  pBLEScan->setActiveScan(true);
  pBLEScan->setInterval(100);
  pBLEScan->setWindow(99);
}

void loop() {
  Serial.println("\n--- SCANNING START ---");

  BLEScanResults* foundDevices = pBLEScan->start(5, fals
e);

  Serial.println("--- SCANNING END ---");

  delay(2000);
}
```

시행착오

시리얼 모니터 속도 115200baud설정 안할시 속도 불일치로 값을 제대로 못읽는다.

```
---- FOUND DEVICE ----
Name: , Address: 2d:17:b3:76:36:0e, rssi: -94, serviceData: J#09DI 2⍰rO0Ó⍰ ⍰⍰⍰y4⍰⍰
---- FOUND DEVICE ----
Name: , Address: 04:8b:96:42:06:99, rssi: -98, serviceData: J#SLEC 2⍰A⍰O⍰⍰⍰⍰k⍰O!⍰⍰&⍰⍰
---- FOUND DEVICE ----
Name: , Address: 1b:94:b1:45:df:ed, rssi: -96, serviceData:  F⍰p⍰;⍰f⍰⍰⍰1⍰
:w⍰⍰⍰
---- FOUND DEVICE ----
Name: , Address: 34:84:fd:53:7b:75, rssi: -94, serviceData: J#XCKU 2⍰' ⍰j⍰4⍰4⍰⍰7k2j⍰⍰
---- FOUND DEVICE ----
Name: , Address: db:99:29:0a:19:41, manufacturer data: 4c0012020002, rssi: -46
---- FOUND DEVICE ----
Name: , Address: 2f:e0:e1:07:3e:67, rssi: -97, serviceData: J#PSWE 2 ⍰"3⍰ih⍰0⍰⍰J⍰⍰f⍰⍰
---- FOUND DEVICE ----
```